# ServiceNow + Microsoft Copilot

# AI Incident Routing

## Complete Implementation Guide

Generated: March 8, 2026

Version 1.0

# Table of Contents


- Project Overview
- System Architecture
- Implementation Guide
- Feasibility Analysis

# Project Overview

# ServiceNow + Microsoft Copilot AI Incident Routing

## 项目完成报告

**项目状态**: **已完成**
**完成时间**: 2026-03-08 13:55 EST
**总耗时**: ~85分钟

---

## 交付内容清单

### 1. 架构设计文档

- **servicenow-copilot-architecture.md** (18.5 KB)
- 系统架构总览 (4层架构图)
- 数据流设计
- 核心组件设计
- 技术选型对比

### 2. 详细实施手册

- **servicenow-copilot-implementation-guide.md** (33.3 KB)
- 环境准备与插件安装
- Azure OpenAI 配置
- CI-KB 关联配置
- AI Routing Engine Script Include (完整代码)
- P1 通知工作流配置
- Copilot 集成步骤
- Virtual Agent Topic 配置
- 测试与验证指南

### 3. 可行性分析报告

- **feasibility-analysis.md** (6.9 KB)
- 技术可行性: **高** □
- 成本可行性: **需谨慎** □
- 实施难度: **中等** □
- 风险评估: **可控** □

### 4. 架构流程图 (6张)

- **01-architecture-overview.jpg** - 整体系统架构
- **02-incident-flow.jpg** - Incident 创建数据流
- **03-copilot-interaction.jpg** - Copilot 交互序列
- **04-rag-process.jpg** - RAG 检索增强流程
- **05-p1-notification.jpg** - P1 通知工作流
- **06-component-relation.jpg** - 组件关系图

### 5. 辅助文件

- **nano-banana-prompts.md** - 图片生成 Prompts
- **project-status.json** - 项目状态跟踪

---

## □ 可行性结论

**总体结论: □ 可行**

### 关键发现

| 维度 | 结论 | 说明 |
|---|---|---|
| **技术可行性** | □ **高** | Yokohama 版本满足要求，原生支持成熟 |
| **成本可行性** | □ **需谨慎** | Now Assist Plus 许可成本 +30-60% |
| **实施难度** | □ **中等** | 4-8 周可完成部署 |
| **风险等级** | □ **可控** | 数据隐私保护完善，有故障回退机制 |

### 针对当前环境的评估

□ **满足的条件**: 1. ServiceNow Yokohama (≥ Xanadu) □ 2. Microsoft 365 Copilot Subscription □ 3. 技术人员到位 □

⚠ **需要确认**: 1. Now Assist Pro Plus 许可状态 2. 预算覆盖成本增幅 (~$20万/年/100用户)

---

## □ 成本估算 (100用户规模)

| 成本项 | 年费用 (USD) |
|---|---|
| ServiceNow ITSM Pro (基础) | $120,000 |
| Now Assist Plus 升级 (+40%) | $48,000 |
| Microsoft 365 Copilot | $36,000 |
| Integration Hub Standard | $1,200 |
| **总计** | **~$205,200** |

---

## □ 实施路线图

| 阶段 | 时间 | 关键任务 |
|---|---|---|
| **Phase 1** | Week 1-2 | 环境准备、插件安装、CI-KB关联 |
| **Phase 2** | Week 3-4 | Azure OpenAI集成、Script开发 |
| **Phase 3** | Week 5-6 | P1通知、On-call配置 |
| **Phase 4** | Week 7-8 | Copilot集成、Virtual Agent配置 |
| **Phase 5** | Week 9-10 | 测试优化、培训上线 |

**总计: 4-8 周**

---

## 文件位置

```
/root/.openclaw/workspace/output/
├── servicenow-copilot-architecture.md      # 架构设计
├── servicenow-copilot-implementation-guide.md  # 实施手册
├── feasibility-analysis.md                 # 可行性分析
├── nano-banana-prompts.md                  # 图片Prompts
├── project-status.json                     # 项目状态
├── diagrams/
│   ├── 01-architecture-overview.jpg        # 架构图
│   ├── 02-incident-flow.jpg                # 流程图
│   ├── 03-copilot-interaction.jpg          # 交互图
│   ├── 04-rag-process.jpg                  # RAG图
│   ├── 05-p1-notification.jpg              # 通知图
│   └── 06-component-relation.jpg           # 组件图
└── SNOW-Copilot-Project.tar.gz             # 完整归档 (532 KB)
```

---

## 如何使用

### 1. 查看文档

```
# 阅读架构设计
cat servicenow-copilot-architecture.md

# 查看实施步骤
cat servicenow-copilot-implementation-guide.md

# 检查可行性
 cat feasibility-analysis.md
```

### 2. 解压归档

```
tar -xzf SNOW-Copilot-Project.tar.gz
```

### 3. 推送到 GitHub

```
# 已在本地初始化Git仓库
cd /root/.openclaw/workspace/output
git remote add origin https://github.com/YOUR_USERNAME/servicenow-copilot-
```

```
routing.git
git push -u origin master
```

---

## 支持资源

| 资源 | 链接 |
|---|---|
| ServiceNow 支持 | support.servicenow.com |
| Microsoft Copilot | docs.microsoft.com/copilot |
| Azure OpenAI | azure.microsoft.com/openai |

---

## 项目统计

- **总文档数**: 5个 Markdown 文件
- **总代码量**: ~500行 Script Include
- **架构图**: 6张高清图片
- **总文件大小**: ~600 KB
- **完成时间**: 85分钟

---

**项目完成** | 富贵虾 (AI Architect) | 2026-03-08

# System Architecture

# ServiceNow + Microsoft Copilot AI Incident Routing 系统

## 整体架构设计方案

---

## 1. 执行摘要

### 1.1 项目背景

当前ServiceNow incident分配依赖于静态规则或人工判断，无法充分利用KB中的处理流程知识。本项目旨在构建一个AI驱动的智能路由系统，能够： - 根据CI关联的KB自动推荐assignment group - 通过Copilot提供自然语言交互界面 - P1 incident自动触发通知机制

### 1.2 核心目标

| 目标 | 描述 |
|---|---|
| **智能路由** | 基于KB内容自动推荐assignment group |
| **AI助手** | 通过Copilot提供交互式建议查询 |
| **快速响应** | P1 incident自动触发多渠道通知 |
| **知识整合** | RAG技术让AI学习企业KB内容 |

---

## 2. 系统架构总览

### 2.1 整体架构图

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          用户交互层 (Presentation Layer)                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────────┐  │
│  │  ServiceNow  │  │ Microsoft    │  │  Virtual Agent / Now Assist  │  │
```

```
│   │  Incident     │  │ Teams         │  │  (自然语言界面)                  │ │
│   │  Form         │  │ Copilot       │  │                                 │ │
│   └───────────────┘  └───────────────┘  └─────────────────────────────────┘ │
└───────────┼──────────────────┼──────────────────────────┼──────────────────────┘
            │                  │                          │
            ▼                  ▼                          ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                        业务逻辑层 (Business Logic Layer)                      │
│   ┌─────────────────┐   ┌──────────────┐   ┌──────────────────────────┐      │
│   │ Business Rule   │   │ AI Routing   │   │ Notification Engine      │      │
│   │ (触发逻辑)       │   │ Engine       │   │ (P1 Alert)               │      │
│   └────────┬────────┘   └──────┬───────┘   └──────────┬───────────────┘      │
│            │                   │                      │                      │
│   ┌────────▼────────┐   ┌──────▼───────┐   ┌──────────▼───────────────┐      │
│   │ Flow Designer   │   │ Script       │   │ On-Call Schedule         │      │
│   │ (Workflow)      │   │ Include      │   │ Integration              │      │
│   └─────────────────┘   └──────────────┘   └──────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────┘
            │
            ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                        AI/ML 层 (AI Service Layer)                           │
│   ┌──────────────────────────────────────────────────────────────────┐       │
│   │              Microsoft Azure OpenAI Service                      │       │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐    │       │
│   │   │ GPT-4/GPT-4o │   │ RAG Engine   │   │ Embeddings       │    │       │
│   │   │ (LLM)        │   │ (KB检索)      │   │ (向量化)          │    │       │
│   │   └──────────────┘   └──────────────┘   └──────────────────┘    │       │
│   └──────────────────────────────────────────────────────────────────┘       │
│                                ▲                                             │
│                                │                                             │
└────────────────────────────────┼─────────────────────────────────────────────┘
                                 │
                                 ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                        数据层 (Data Layer)                                    │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐  │
│   │ Incident     │   │ KB Articles  │   │ CMDB (CI)    │   │ AI Search    │  │
│   │ Table        │   │ (处理流程)    │   │ (support_grp)│   │ Index        │  │
│   └──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘  │
└──────────────────────────────────────────────────────────────────────────────┘
```

## 2.2 数据流架构

```
Incident Creation Flow:
──────────────────────────────────────────────────────────────────────────────


User creates Incident
```

```
        │
        ▼
┌─────────────────┐
│ Business Rule   │──[1]──▶ Check if cmdb_ci is set
│ (After Insert)  │
└─────────────────┘
        │
        ▼
┌─────────────────┐
│ KB Retrieval    │──[2]──▶ Query KB articles related to CI
│ (RAG - Step 1)  │         (use AI Search or scripted query)
└─────────────────┘
        │
        ▼
┌─────────────────┐
│ AI Analysis     │──[3]──▶ Send KB content + Incident details
│ (Azure OpenAI)  │         to LLM for assignment recommendation
└─────────────────┘
        │
        ▼
┌─────────────────┐
│ Parse Response  │──[4]──▶ Extract assignment_group from AI response
│ (Script Include)│
└─────────────────┘
        │
        ▼
┌─────────────────┐
│ Update Incident │──[5]──▶ Set assignment_group
│ (Set Values)    │         Add work note with AI reasoning
└─────────────────┘
        │
        ▼
┌─────────────────┐
│ Priority Check  │──[6]──▶ If Priority = 1 (P1)
│ (Condition)     │         Trigger notification workflow
└─────────────────┘
        │
    ┌───┴───┐
    ▼       ▼
  P1=Y    P1=N
    │       │
    ▼       │
┌─────────────────┐
│ P1 Notification │──[7]──▶ Send SMS/Email/Teams notification
│ Workflow        │         to on-call engineer + manager
└─────────────────┘

Copilot Interaction Flow:
────────────────────────────────────────────────────────────────────────────
```

```
Support Engineer opens Teams Copilot
        │
        ▼
┌──────────────────┐
│ Natural Lang     │──[8]──▶ "Who should handle this incident?"
│ Query Input      │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Copilot routes   │──[9]──▶ Intent recognition (Now Assist NLU)
│ to Now Assist    │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Virtual Agent    │──[10]─▶ Fetch incident details
│ Topic Execution  │         Query KB for similar cases
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ AI generates     │──[11]─▶ Azure OpenAI generates recommendation
│ recommendation   │         with explanation
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Display in       │──[12]─▶ Show recommendation + confidence score
│ Teams Chat       │         + option to apply assignment
└──────────────────┘
```

---

# 3. 核心组件设计

## 3.1 CI-KB关联模型

```
// CI-KB关联数据结构
{
    "cmdb_ci": {
        "sys_id": "unique_ci_id",
        "name": "SAP Production Server",
        "support_group": "SAP Team (sys_id)",
        "kb_articles": [
            {
                "number": "KB0012345",
                "title": "SAP Server Incident Handling",
```

```
                "category": "escalation_procedure",
                "assignment_rules": {
                    "network_issue": "Network Team",
                    "db_issue": "DBA Team",
                    "app_issue": "SAP Team"
                }
            }
        ]
    }
}
```

## 3.2 AI路由引擎设计

```
// Script Include: AIRoutingEngine
var AIRoutingEngine = Class.create();
AIRoutingEngine.prototype = {
    initialize: function() {
        this.azureEndpoint = gs.getProperty('azure.openai.endpoint');
        this.apiKey = gs.getProperty('azure.openai.key');
        this.deploymentName = gs.getProperty('azure.openai.deployment');
    },

    // 主函数：获取assignment group推荐
    getAssignmentRecommendation: function(incidentId) {
        var incident = new GlideRecord('incident');
        if (!incident.get(incidentId)) return null;

        // 1. 收集上下文信息
        var context = this._gatherContext(incident);

        // 2. 检索相关KB
        var kbContent = this._retrieveKBContent(context.ciSysId);

        // 3. 构建prompt
        var prompt = this._buildPrompt(incident, context, kbContent);

        // 4. 调用Azure OpenAI
        var aiResponse = this._callAzureOpenAI(prompt);

        // 5. 解析响应
        return this._parseAIResponse(aiResponse);
    },

    // RAG: 检索增强生成
    _retrieveKBContent: function(ciSysId) {
        // 使用ServiceNow AI Search或直接查询
        var kbGR = new GlideRecord('kb_knowledge');
        kbGR.addQuery('cmdb_ci', ciSysId);
```

```
        kbGR.addActiveQuery();
        kbGR.query();

        var articles = [];
        while (kbGR.next()) {
            articles.push({
                number: kbGR.getValue('number'),
                title: kbGR.getValue('short_description'),
                content: kbGR.getValue('text')
            });
        }
        return articles;
    },

    _buildPrompt: function(incident, context, kbContent) {
        return {
            system: "You are an expert ITSM routing assistant. Based on the
incident details and KB articles, recommend the best assignment group.",
            user: {
                incident: {
                    short_description: incident.short_description,
                    description: incident.description,
                    category: incident.category,
                    subcategory: incident.subcategory,
                    ci_name: context.ciName,
                    urgency: incident.urgency,
                    impact: incident.impact
                },
                kb_articles: kbContent,
                available_groups: context.availableGroups
            },
            output_format: {
                assignment_group: "recommended_group_name",
                confidence: "high/medium/low",
                reasoning: "explanation"
            }
        };
    }
};
```

## 3.3 P1通知工作流

```
P1 Notification Workflow:
┌──────────────────┐
│ Trigger:         │
│ Priority = 1     │
│ AND              │
│ State = New      │
```

```
└──────────────┘
        │
        ▼
┌──────────────┐
│ Action 1:    │
│ Get On-Call  │
│ Engineer     │
│ (from Schedule) │
└──────────────┘
        │
        ▼
┌──────────────┐
│ Action 2:    │
│ Send Email   │
│ (Manager DL) │
└──────────────┘
        │
        ▼
┌──────────────┐
│ Action 3:    │
│ Send SMS     │
│ (On-Call Phone) │
└──────────────┘
        │
        ▼
┌──────────────┐
│ Action 4:    │
│ Teams Message │
│ (Escalation Ch) │
└──────────────┘
```

---

# 4. Microsoft Copilot 集成方案

## 4.1 集成架构

| 组件 | 功能 | 配置要点 |
|---|---|---|
| **Custom Engine Agent** | Copilot与ServiceNow的桥梁 | Zurich+版本内置支持 |
| **Now Assist** | ServiceNow的AI平台 | 需Now Assist许可证 |
| **Generative AI Controller** | 连接Azure OpenAI | 配置API endpoint和key |
| **Virtual Agent** | 对话界面 | Topic设计和NLU训练 |

## 4.2 Copilot交互场景

### 场景1: Assignment建议查询

```
User: "Who should handle INC0012345?"
Copilot: [识别intent] → 调用Now Assist
Now Assist: [查询incident] → 获取CI和KB信息
Now Assist: [调用Azure OpenAI] → 生成推荐
Copilot: "Based on the KB article KB0012345, this SAP server issue
          should be assigned to the SAP Team.
          Confidence: High
          Would you like me to update the assignment?"
```

### 场景2: 历史案例查询

```
User: "Have we seen this issue before?"
Copilot: [搜索similar incidents] → 返回历史案例
Copilot: "Found 3 similar incidents in the past 30 days.
          Resolution: Restart SAP service."
```

---

# 5. 实施路线图

## 5.1 阶段划分

| 阶段 | 时间 | 交付物 | 负责方 |
|---|---|---|---|
| **Phase 1** | Week 1-2 | 基础配置：CI-KB关联、Assignment规则 | @shirmpclaw |
| **Phase 2** | Week 3-4 | Azure OpenAI集成、Script Include开发 | @shirmpclaw |
| **Phase 3** | Week 5-6 | P1通知工作流、On-call schedule集成 | @shirmpclaw |
| **Phase 4** | Week 7-8 | Copilot集成、Virtual Agent配置 | @shirmpclaw |
| **Phase 5** | Week 9-10 | 测试优化、培训、上线 | 联合 |

## 5.2 依赖关系

```
[CI数据清理] ──────┐
                   ▼
[KB标准化] ──→ [AI Routing Engine] ──→ [Copilot Integration]
                   ▲                          ▲
```

```
[Azure OpenAI] ──┘                          │
                                            │
[P1 Notification] ──────────────────────────┘
```

---

## 6. 风险评估与缓解

| 风险 | 影响 | 缓解措施 |
|---|---|---|
| AI推荐不准确 | 高 | 保留人工覆盖机制，设置confidence阈值 |
| Azure OpenAI延迟 | 中 | 异步处理，设置timeout，fallback到静态规则 |
| P1通知失败 | 高 | 多渠道冗余(SMS+Email+Teams) |
| 数据隐私 | 高 | PII脱敏，限制AI访问范围 |

---

## 7. 附录

### 附录A: 需要的ServiceNow插件

- Virtual Agent
- Now Assist
- Conversational Integration with Microsoft Teams
- Integration Hub (for P1 notifications)

### 附录B: Azure资源需求

- Azure OpenAI Service (GPT-4/GPT-4o)
- Azure AI Search (for RAG indexing)
- (可选) Azure Functions (for webhook处理)

---

文档生成时间: 2026-03-08 版本: v1.0 作者: 富贵虾 (AI Architect) + @shirmpclaw (Implementation)

# Implementation Guide

# ServiceNow + Microsoft Copilot AI Incident Routing

## 详细配置实施手册

# 第一部分：环境准备与前置配置

## 1.1 ServiceNow 插件安装与配置

### 1.1.1 必需插件清单

登录 ServiceNow 实例，导航至 **System Applications → All Available Applications → All**，安装以下插件：

| 插件名称 | 插件ID | 用途 | 状态 |
|---|---|---|---|
| Now Assist | com.snc.now_assist | AI核心平台 | 必需 |
| Virtual Agent | com.glide.virtual_agent | 对话机器人 | 必需 |
| IntegrationHub | com.glide.hub.integrations | 集成平台 | 必需 |
| AI Search | com.snc.ai_search | 语义搜索 | 必需 |
| Microsoft Teams Integration | com.snc.teams_integration | Teams连接器 | 必需 |
| Flow Designer | com.glide.flow_designer | 工作流设计 | 必需 |

**安装步骤：**

```
1. 导航至: All > System Applications > All Available Applications > All
2. 搜索 "Now Assist" → 点击 Install
3. 等待安装完成（约5-10分钟）
4. 重复上述步骤安装其他插件
```

### 1.1.2 系统属性配置

导航至 **System Properties → All Properties**，配置以下属性：

| 属性名称 | 值 | 说明 |
|---|---|---|
| `glide.virtual_agent.nlu.enabled` | `true` | 启用NLU |
| `glide.now.assist.enabled` | `true` | 启用Now Assist |
| `glide.ai_search.enabled` | `true` | 启用AI Search |

## 1.2 Azure OpenAI 服务配置

### 1.2.1 Azure 门户配置

**步骤 1**: 创建 Azure OpenAI Service 资源

```
1. 登录 Azure Portal (https://portal.azure.com)
2. 点击 "Create a resource"
3. 搜索 "Azure OpenAI" → 点击 Create
4. 填写基本信息：
   - Subscription: 选择订阅
   - Resource group: 创建或使用现有 (如: servicenow-ai)
   - Region: East US / West Europe (根据就近原则)
   - Name: servicenow-openai-prod
   - Pricing tier: Standard S0
5. 点击 Review + Create → Create
```

**步骤 2**: 部署模型

```
1. 进入创建的 Azure OpenAI 资源
2. 点击 "Model deployments" → "Create new deployment"
3. 选择模型: GPT-4 或 GPT-4o
4. 部署名称: gpt-4-servicenow
5. 版本: 选择最新稳定版
6. 点击 Create
```

**步骤 3**: 获取 API Key 和 Endpoint

```
1. 在 Azure OpenAI 资源页面，点击 "Keys and Endpoint"
2. 复制以下信息（保存到安全位置）：
   - KEY 1: sk-xxxxxxxxxxxxxxxxxxxxxxxx
   - Endpoint: https://servicenow-openai-prod.openai.azure.com/
   - Deployment name: gpt-4-servicenow
```

### 1.2.2 ServiceNow 中配置 Azure OpenAI 连接

**步骤 1**: 创建 REST Message

```
导航: All > System Web Services > REST Message
点击: New
```

填写以下信息：

| 字段 | 值 |
| --- | --- |
| Name | Azure OpenAI Service |
| Endpoint | https://servicenow-openai-prod.openai.azure.com |
| Authentication | No authentication (在HTTP Request中使用Header) |

**步骤 2**: 配置 HTTP Method

在 REST Message 记录中，滚动到 **HTTP Methods** 相关列表，点击 **New**：

```
Name: chat_completion
HTTP Method: POST
Endpoint: /openai/deployments/gpt-4-servicenow/chat/completions?api-
version=2024-02-15-preview
```

**步骤 3**: 配置 HTTP Headers

在 HTTP Method 记录中，添加 Headers：

| Header Name | Value |
| --- | --- |
| Content-Type | application/json |
| api-key | ${azure.api.key} |

**步骤 4**: 配置 Request Body

```
{
  "messages": [
    {
      "role": "system",
      "content": "${system_prompt}"
    },
    {
      "role": "user",
      "content": "${user_prompt}"
    }
  ],
  "temperature": 0.3,
  "max_tokens": 800
}
```

**步骤 5**: 配置 Variables

在 REST Message 中定义变量：

| Variable Name | Value |
| --- | --- |
| azure.api.key | [从Azure复制的Key] |
| system_prompt | You are an ITSM routing assistant... |
| user_prompt | [动态传入] |

---

# 第二部分：CI-KB 关联与数据模型配置

## 2.1 CMDB CI 字段扩展

### 2.1.1 添加 Support Group 字段到 CI

**步骤 1**: 导航至 **System Definition → Tables**

**步骤 2**: 搜索表 **Configuration Item [cmdb_ci]**

**步骤 3**: 在 **Columns** 相关列表中，点击 **New**：

```
Column Label: Support Group
Column Name: support_group
Type: Reference
Reference: Group [sys_user_group]
Max length: 32
Active: true
```

**步骤 4**: 添加表单字段

```
导航: All > Configuration > Servers > All Servers (或其他CI类型)
选择任意CI记录 → 右键 Form > Configure > Form Layout
将 "Support Group" 字段从左侧拖到右侧表单
保存
```

### 2.1.2 批量导入 CI-Support Group 映射

**准备导入数据** (Excel/CSV)：

| cmdb_ci.name | support_group.name |
|---|---|
| SAP Production Server | SAP Team |
| Oracle Database Server | DBA Team |
| Network Switch 01 | Network Operations |

**导入步骤**：

```
1. 导航: All > System Import Sets > Load Data
2. Source: File (上传CSV)
3. Target Table: Configuration Item [cmdb_ci]
4. Transform Map: 创建新的Transform Map
   - Source field: cmdb_ci.name → Target: name
   - Source field: support_group.name → Target: support_group (使用Reference
Qualifier)
5. 运行Transform
```

---

## 2.2 KB 与 CI 关联配置

### 2.2.1 启用 CI 关联功能

**步骤 1**: 导航 **Knowledge → Administration → Properties**

**步骤 2**: 启用以下属性：

```
glide.know.use.cmdb_ci = true
glide.know.show.cmdb_ci = true
```

### 2.2.2 配置 KB 表单

**步骤 1**: 导航 **Knowledge → All**

**步骤 2**: 打开任意 KB Article → 右键 **Configure → Form Layout**

**步骤 3**: 添加字段： - **Configuration Item**: 从左侧拖到右侧 - **Assignment Group**: 拖到右侧（如果还没有）

### 2.2.3 创建 KB 分类标准

**步骤 1**: 导航 **Knowledge → Administration → Categories**

**步骤 2**: 创建以下分类结构：

```
IT Operations
├── Server Issues
│   ├── Hardware Failures
│   └── OS Problems
├── Database Issues
│   ├── Oracle
│   └── SQL Server
```

```
├── Network Issues
│   ├── Connectivity
│   └── Performance
└── Application Issues
    ├── SAP
    └── Custom Apps
```

**步骤 3**: 为每个分类配置 **Owner Group**（用于默认assignment）

---

# 第三部分：AI 路由引擎开发

## 3.1 Script Include: AIRoutingEngine

### 3.1.1 创建 Script Include

**步骤 1**: 导航 **System Definition → Script Includes → New**

**步骤 2**: 填写基本信息：

```
Name: AIRoutingEngine
API Name: AIRoutingEngine
Client callable: false
Active: true
Application scope: Global
```

**步骤 3**: 粘贴以下代码：

```
var AIRoutingEngine = Class.create();
AIRoutingEngine.prototype = {
    initialize: function() {
        this.restMessageName = 'Azure OpenAI Service';
        this.httpMethod = 'chat_completion';
        this.maxRetries = 3;
        this.timeout = 30000; // 30 seconds
    },

    /**
     * 主函数：获取 Assignment Group 推荐
     * @param {string} incidentId - Incident sys_id
     * @returns {Object} recommendation object
     */
    getAssignmentRecommendation: function(incidentId) {
        try {
            // 1. 获取 Incident 数据
            var incident = this._getIncidentData(incidentId);
            if (!incident) {
                return this._createErrorResponse('Incident not found');
            }

            // 2. 检索相关 KB Articles
            var kbArticles = this._retrieveKBArticles(incident.cmdb_ci);
```

```
        // 3. 构建 Prompt
        var prompt = this._buildPrompt(incident, kbArticles);

        // 4. 调用 Azure OpenAI
        var aiResponse = this._callAzureOpenAI(prompt);
        if (aiResponse.error) {
            return this._createErrorResponse(aiResponse.error);
        }

        // 5. 解析响应
        var recommendation = this._parseAIResponse(aiResponse, incident);

        // 6. 记录日志
        this._logRecommendation(incidentId, recommendation);

        return recommendation;

    } catch (e) {
        gs.error('AIRoutingEngine Error: ' + e.message);
        return this._createErrorResponse(e.message);
    }
},

/**
 * 获取 Incident 完整数据
 */
_getIncidentData: function(incidentId) {
    var gr = new GlideRecord('incident');
    if (!gr.get(incidentId)) {
        return null;
    }

    var ciSysId = gr.getValue('cmdb_ci');
    var ciName = '';
    var supportGroup = '';

    if (ciSysId) {
        var ci = new GlideRecord('cmdb_ci');
        if (ci.get(ciSysId)) {
            ciName = ci.getValue('name');
            var sg = ci.getValue('support_group');
            if (sg) {
                var group = new GlideRecord('sys_user_group');
                if (group.get(sg)) {
                    supportGroup = group.getValue('name');
                }
            }
        }
    }
```

```
        return {
            sys_id: incidentId,
            number: gr.getValue('number'),
            short_description: gr.getValue('short_description'),
            description: gr.getValue('description'),
            category: gr.getValue('category'),
            subcategory: gr.getValue('subcategory'),
            priority: gr.getValue('priority'),
            urgency: gr.getValue('urgency'),
            impact: gr.getValue('impact'),
            cmdb_ci: ciSysId,
            ci_name: ciName,
            ci_support_group: supportGroup,
            assigned_to: gr.getValue('assigned_to'),
            assignment_group: gr.getValue('assignment_group')
        };
    },

    /**
     * RAG: 检索KB Articles
     */
    _retrieveKBArticles: function(ciSysId) {
        var articles = [];

        if (!ciSysId) {
            return articles;
        }

        // 方法1: 通过AI Search (如果可用)
        try {
            articles = this._searchWithAISearch(ciSysId);
        } catch (e) {
            gs.warn('AI Search failed, falling back to direct query: ' +
e.message);
            articles = this._searchWithDirectQuery(ciSysId);
        }

        return articles;
    },

    /**
     * 使用AI Search (Semantic Search)
     */
    _searchWithAISearch: function(ciSysId) {
        var articles = [];

        // 获取CI详细信息用于语义搜索
        var ciInfo = this._getCIInfoForSearch(ciSysId);
```

```
        // 使用ServiceNow AI Search API
        var searchRequest = {
            search_terms: ciInfo.name + ' ' + ciInfo.model_id + ' incident
handling',
            table: 'kb_knowledge',
            limit: 3,
            semantic_search: true
        };

        // 注意: 实际实现可能需要使用Scripted REST API 或Flow Designer
        // 这里使用简化版本
        var gr = new GlideRecord('kb_knowledge');
        gr.addQuery('cmdb_ci', ciSysId);
        gr.addQuery('workflow_state', 'published');
        gr.addActiveQuery();
        gr.orderByDesc('sys_updated_on');
        gr.setLimit(3);
        gr.query();

        while (gr.next()) {
            articles.push({
                number: gr.getValue('number'),
                title: gr.getValue('short_description'),
                content: gr.getValue('text').substring(0, 1000), // 限制长度
                category: gr.getDisplayValue('kb_category'),
                sys_id: gr.getValue('sys_id')
            });
        }

        return articles;
    },

    /**
     * 直接查询 (Fallback)
     */
    _searchWithDirectQuery: function(ciSysId) {
        var articles = [];

        var gr = new GlideRecord('kb_knowledge');
        gr.addQuery('cmdb_ci', ciSysId);
        gr.addQuery('workflow_state', 'published');
        gr.addActiveQuery();
        gr.orderByDesc('sys_updated_on');
        gr.setLimit(3);
        gr.query();

        while (gr.next()) {
            articles.push({
                number: gr.getValue('number'),
                title: gr.getValue('short_description'),
```

```
                content: gr.getValue('text').substring(0, 1000),
                category: gr.getDisplayValue('kb_category'),
                sys_id: gr.getValue('sys_id')
            });
        }

        return articles;
    },

    /**
     * 获取CI 信息用于搜索
     */
    _getCIInfoForSearch: function(ciSysId) {
        var info = {
            name: '',
            model_id: '',
            class_name: ''
        };

        var gr = new GlideRecord('cmdb_ci');
        if (gr.get(ciSysId)) {
            info.name = gr.getValue('name');
            info.model_id = gr.getValue('model_id');
            info.class_name = gr.getClassDisplayValue();
        }

        return info;
    },

    /**
     * 构建Prompt
     */
    _buildPrompt: function(incident, kbArticles) {
        var systemPrompt = "You are an expert IT Service Management routing
assistant. " +
            "Analyze the incident details and KB articles to recommend the best
assignment group. " +
            "Consider the CI type, incident category, and KB article guidance. " +
            "Respond in JSON format with: assignment_group, confidence (high/
medium/low), and reasoning.";

        var userPrompt = "Incident Details:\n" +
            "Number: " + incident.number + "\n" +
            "Short Description: " + incident.short_description + "\n" +
            "Description: " + incident.description + "\n" +
            "Category: " + incident.category + "\n" +
            "Subcategory: " + incident.subcategory + "\n" +
            "Priority: " + incident.priority + "\n" +
            "Affected CI: " + incident.ci_name + "\n" +
            "CI Default Support Group: " + incident.ci_support_group + "\n\n";
```

```
        if (kbArticles.length > 0) {
            userPrompt += "Relevant KB Articles:\n";
            for (var i = 0; i < kbArticles.length; i++) {
                userPrompt += (i + 1) + ". " + kbArticles[i].number + " - " +
kbArticles[i].title + "\n";
                userPrompt += "Category: " + kbArticles[i].category + "\n";
                userPrompt += "Content: " + kbArticles[i].content.substring(0,
500) + "...\n\n";
            }
        }

        userPrompt += "\nBased on this information, recommend the assignment group
in this JSON format:\n" +
            "{\n" +
            "  \"assignment_group\": \"Group Name\",\n" +
            "  \"confidence\": \"high/medium/low\",\n" +
            "  \"reasoning\": \"explanation\"\n" +
            "}";

        return {
            system: systemPrompt,
            user: userPrompt
        };
    },

    /**
     * 调用Azure OpenAI
     */
    _callAzureOpenAI: function(prompt) {
        var r = new sn_ws.RESTMessageV2(this.restMessageName, this.httpMethod);

        // 设置变量
        r.setStringParameter('system_prompt', prompt.system);
        r.setStringParameter('user_prompt', prompt.user);

        // 设置超时
        r.setHttpTimeout(this.timeout);

        var response;
        var retries = 0;

        while (retries < this.maxRetries) {
            try {
                response = r.execute();
                var httpStatus = response.getStatusCode();

                if (httpStatus == 200) {
                    var body = response.getBody();
                    return JSON.parse(body);
```

```
                } else {
                    retries++;
                    if (retries >= this.maxRetries) {
                        return { error: 'HTTP ' + httpStatus + ': ' +
response.getErrorMessage() };
                    }
                    // 等待1秒后重试
                    this._sleep(1000);
                }
            } catch (e) {
                retries++;
                if (retries >= this.maxRetries) {
                    return { error: e.message };
                }
                this._sleep(1000);
            }
        }

        return { error: 'Max retries exceeded' };
    },

    /**
     * 解析AI 响应
     */
    _parseAIResponse: function(aiResponse, incident) {
        try {
            // 提取content 从OpenAI 响应
            var content = '';
            if (aiResponse.choices && aiResponse.choices.length > 0) {
                content = aiResponse.choices[0].message.content;
            }

            // 尝试解析JSON
            var parsed;
            try {
                parsed = JSON.parse(content);
            } catch (e) {
                // 如果不是纯JSON，尝试提取JSON部分
                var jsonMatch = content.match(/\{[\s\S]*\}/);
                if (jsonMatch) {
                    parsed = JSON.parse(jsonMatch[0]);
                } else {
                    throw new Error('Invalid JSON response');
                }
            }

            // 验证assignment_group
            var groupSysId = this._validateAndGetGroup(parsed.assignment_group);

            return {
```

```
                success: true,
                assignment_group_name: parsed.assignment_group,
                assignment_group_sysid: groupSysId,
                confidence: parsed.confidence || 'medium',
                reasoning: parsed.reasoning || 'No reasoning provided',
                original_ci_support_group: incident.ci_support_group,
                ai_recommended: groupSysId !== incident.assignment_group
            };

        } catch (e) {
            return this._createErrorResponse('Failed to parse AI response: ' +
e.message);
        }
    },

    /**
     * 验证并获取Group Sys ID
     */
    _validateAndGetGroup: function(groupName) {
        if (!groupName) return null;

        var gr = new GlideRecord('sys_user_group');
        gr.addQuery('name', groupName);
        gr.addActiveQuery();
        gr.query();

        if (gr.next()) {
            return gr.getValue('sys_id');
        }

        return null;
    },

    /**
     * 记录推荐日志
     */
    _logRecommendation: function(incidentId, recommendation) {
        gs.info('AI Routing Recommendation for ' + incidentId + ': ' +
            JSON.stringify(recommendation));

        // 可以扩展到写入自定义日志表
    },

    /**
     * 创建错误响应
     */
    _createErrorResponse: function(errorMessage) {
        return {
            success: false,
            error: errorMessage,
```

```
            assignment_group_name: null,
            assignment_group_sysid: null,
            confidence: 'low',
            reasoning: 'AI recommendation failed: ' + errorMessage
        };
    },

    /**
     * 辅助：睡眠函数
     */
    _sleep: function(ms) {
        var end = new Date().getTime() + ms;
        while (new Date().getTime() < end) {
            // 忙等待
        }
    },

    type: 'AIRoutingEngine'
};
```

**步骤 4**: 点击 **Submit** 保存

---

# 3.2 Business Rule: 自动触发 AI 路由

## 3.2.1 创建 After Insert Business Rule

**步骤 1**: 导航 **System Definition → Business Rules → New**

**步骤 2**: 填写基本信息：

```
Name: AI Routing - Auto Assignment
Table: Incident [incident]
Active: true
Advanced: true
When: after
Insert: true
Update: false
Filter Conditions:
  - Configuration item is not empty
  - Assignment group is empty
```

**步骤 3**: 在 **Advanced** 标签页粘贴脚本：

```
(function executeRule(current, previous /*null when async*/) {

    // 避免递归触发
    if (current.getValue('u_ai_processing') == 'true') {
        return;
    }

    try {
        // 调用AI 路由引擎
        var aiRouter = new AIRoutingEngine();
        var recommendation =
aiRouter.getAssignmentRecommendation(current.getValue('sys_id'));

        if (recommendation.success && recommendation.assignment_group_sysid) {

            // 设置标记避免递归
            current.setValue('u_ai_processing', 'true');

            // 更新Assignment Group
            current.setValue('assignment_group',
recommendation.assignment_group_sysid);

            // 添加工作记录
            var workNote = '[AI Routing] Recommended assignment: ' +
                recommendation.assignment_group_name +
                ' (Confidence: ' + recommendation.confidence + ')\n' +
                'Reasoning: ' + recommendation.reasoning;

            current.setWorkNote(workNote);

            gs.info('AI Routing applied to ' + current.getValue('number') +
                ': ' + recommendation.assignment_group_name);
        } else {
            // AI 推荐失败，使用CI 默认Support Group
            var ciSysId = current.getValue('cmdb_ci');
            if (ciSysId) {
                var ci = new GlideRecord('cmdb_ci');
                if (ci.get(ciSysId) && ci.getValue('support_group')) {
                    current.setValue('assignment_group',
ci.getValue('support_group'));
                    current.setWorkNote('[AI Routing] Fallback to CI Support
Group. AI Error: ' +
                        (recommendation.error || 'Unknown'));
                }
            }
        }

    } catch (e) {
        gs.error('AI Routing Business Rule Error: ' + e.message);
```

```
    }

})(current, previous);
```

**步骤 4**: 点击 **Submit**

### 3.2.2 创建字段标记

**步骤 1**: 导航 **System Definition → Tables → Incident**

**步骤 2**: 在 **Columns** 中新建：

```
Column Label: AI Processing
Column Name: u_ai_processing
Type: True/False
Default value: false
Active: true
```

---

# 第四部分：P1 通知工作流配置

## 4.1 Notification Rules 配置

### 4.1.1 创建 P1 Incident Notification

**步骤 1**: 导航 **System Notification → Notifications → New**

**步骤 2**: 填写基本信息：

```
Name: P1 Incident Alert - On-Call Engineer
Table: Incident [incident]
Active: true
Category: Incident Management
```

**步骤 3**: 配置 **When to send**：

```
Send when: Record inserted or updated
Condition: Priority is 1 - Critical
           AND State is New
```

**步骤 4**: 配置 **Who will receive**：

```
User/Group: [Script]
```

在 **Recipient Script** 中：

```
(function runRecipientScript(current) {
    var recipients = [];

    // 获取Assignment Group 的On-Call 成员
    var groupId = current.getValue('assignment_group');
    if (groupId) {
        // 使用On-Call Scheduling API
        var onCallGR = new GlideRecord('cmn_rota');
        onCallGR.addQuery('group', groupId);
        onCallGR.addActiveQuery();
        onCallGR.query();

        while (onCallGR.next()) {
```

```
            var member = onCallGR.getValue('member');
            if (member) {
                recipients.push(member);
            }
        }
    }

    // 如果找不到on-call，发送给group manager
    if (recipients.length === 0 && groupId) {
        var group = new GlideRecord('sys_user_group');
        if (group.get(groupId) && group.getValue('manager')) {
            recipients.push(group.getValue('manager'));
        }
    }

    return recipients;
})(current);
```

### 4.1.2 配置通知内容

**步骤 1**: 在 **What it will contain** 中：

```
Subject: [P1 CRITICAL] ${number} - ${short_description}

Message HTML:
```

```
<div style="font-family: Arial, sans-serif; padding: 20px;">
    <div style="background-color: #d32f2f; color: white; padding: 15px; border-
radius: 5px;">
        <h2 style="margin: 0;">□ P1 CRITICAL INCIDENT</h2>
    </div>

    <div style="margin-top: 20px;">
        <p><strong>Incident Number:</strong> ${number}</p>
        <p><strong>Short Description:</strong> ${short_description}</p>
        <p><strong>Priority:</strong> ${priority}</p>
        <p><strong>Affected CI:</strong> ${cmdb_ci}</p>
        <p><strong>Assignment Group:</strong> ${assignment_group}</p>
        <p><strong>Opened:</strong> ${opened_at}</p>
        <p><strong>Opened By:</strong> ${opened_by}</p>
    </div>

    <div style="margin-top: 20px; padding: 15px; background-color: #fff3e0;
border-left: 4px solid #ff9800;">
        <p><strong>Description:</strong></p>
        <p>${description}</p>
    </div>
```

```
    <div style="margin-top: 20px;">
        <a href="${instance_url}/nav_to.do?uri=incident.do?sys_id=${sys_id}"
           style="background-color: #1976d2; color: white; padding: 10px 20px;
text-decoration: none; border-radius: 5px;">
            View Incident
        </a>
    </div>
</div>
```

---

# 4.2 Flow Designer: P1 Escalation Workflow

## 4.2.1 创建 Flow

**步骤 1**: 导航 **Flow Designer**

**步骤 2**: 点击 **New → Flow**

```
Flow Name: P1 Incident Escalation
Run: Always
Run As: System User
```

## 4.2.2 配置 Trigger

**添加 Trigger**: - **Table**: Incident - **Condition**: Priority is 1 - Critical - **Run**: When record is created

## 4.2.3 添加 Actions

**Action 1**: Wait for duration

```
Duration: 15 minutes
```

**Action 2**: Check if incident is still open

```
Condition: State is not Resolved
          AND State is not Closed
```

**Action 3**: If condition true - Escalate

```
Action: Send Email
To: [Assignment Group Manager]
Subject: [ESCALATION] P1 ${number} unresolved after 15 minutes
```

**Action 4**: Create Slack Message

```
Action: Slack - Post Message
Channel: #incidents-critical
Message: □ P1 Escalation: ${number} needs attention
```

**步骤 4**: 保存并激活 Flow

---

# 第五部分：Microsoft Copilot 集成

## 5.1 Azure AD 应用注册

### 5.1.1 创建应用注册

**步骤 1**: 登录 **Azure Portal → Azure Active Directory → App registrations → New registration**

**步骤 2**: 填写信息：

```
Name: ServiceNow-Copilot-Integration
Supported account types: Accounts in this organizational directory only
Redirect URI (optional): https://your-instance.service-now.com/oauth_redirect.do
```

**步骤 3**: 点击 **Register**

**步骤 4**: 记录以下信息：

```
Application (client) ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Directory (tenant) ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

### 5.1.2 添加客户端密钥

**步骤 1**: 在应用页面，点击 **Certificates & secrets → New client secret**

**步骤 2**: 填写描述并选择过期时间

**步骤 3**: 立即复制密钥值（只显示一次）

### 5.1.3 配置 API 权限

**步骤 1**: 点击 **API permissions → Add a permission**

**步骤 2**: 添加以下权限：

```
Microsoft Graph:
  - User.Read
  - Chat.Read
```

```
  - Chat.Create
  - TeamsAppInstallation.ReadForUser

Azure Service Management:
  - user_impersonation
```

**步骤 3**: 点击 **Grant admin consent**

---

# 5.2 ServiceNow OAuth 配置

## 5.2.1 创建 OAuth Provider

**步骤 1**: 导航 **System OAuth → Application Registry → New**

**步骤 2**: 选择 **Connect to a third party OAuth Provider**

**步骤 3**: 填写信息：

```
Name: Microsoft Teams Copilot
Client ID: [从Azure复制的Client ID]
Client Secret: [从Azure复制的Client Secret]
Token URL: https://login.microsoftonline.com/[tenant-id]/oauth2/v2.0/token
Authorization URL: https://login.microsoftonline.com/[tenant-id]/oauth2/v2.0/
authorize
Default Grant Type: Authorization Code
Redirect URL: https://your-instance.service-now.com/oauth_redirect.do
```

---

# 5.3 Teams Integration 配置

## 5.3.1 安装 Teams Integration 插件

已在 **1.1.1** 中安装 `com.snc.teams_integration`

## 5.3.2 配置 Teams 连接

**步骤 1**: 导航 **Microsoft Teams Integration → Setup**

**步骤 2**: 点击 **Configure**

**步骤 3**: 授权 Teams 连接

### 5.3.3 创建 Custom Engine Agent (Zurich+)

**注意**: 如果实例是 **Zurich 版本**，可以使用 Custom Engine Agent：

**步骤 1**: 导航 **Conversational Interfaces → Settings → Microsoft Copilot**

**步骤 2**: 启用 **Custom Engine Agent**

**步骤 3**: 配置 Bot 名称和 Manifest

---

## 5.4 Virtual Agent Topic 配置

### 5.4.1 创建 NLU Model

**步骤 1**: 导航 **NLU Workbench → Models → New**

```
Name: Incident Routing Model
Language: English (or your language)
```

**步骤 2**: 创建 Intents：

| Intent | Description | Sample Utterances |
|---|---|---|
| assignment_query | 查询assignment建议 | "Who should handle this incident?", "What team should I assign this to?" |
| kb_search | 搜索KB | "Find KB articles for this issue", "Show me documentation" |
| escalate_incident | 升级incident | "I need to escalate this", "This is urgent" |

### 5.4.2 创建 Virtual Agent Topic

**步骤 1**: 导航 **Virtual Agent → Topics → New**

```
Name: Incident Assignment Assistant
NLU Model: Incident Routing Model
Trigger Intent: assignment_query
```

**步骤 2**: 配置 Topic Flow：

```
1. Start
2. Ask for: Incident Number (or use current context)
3. Script Action: Call AIRoutingEngine.getAssignmentRecommendation()
4. Display: AI recommendation with confidence
5. Ask: "Would you like me to update the assignment?"
6. If Yes → Update Incident → Confirmation message
7. If No → End
```

**步骤 3**: 在 **Script Action** 中：

```
(function execute() {
    var incidentNumber = conversation.variables.incident_number;

    // 获取incident sys_id
    var gr = new GlideRecord('incident');
    gr.addQuery('number', incidentNumber);
    gr.query();

    if (gr.next()) {
        var aiRouter = new AIRoutingEngine();
        var rec = aiRouter.getAssignmentRecommendation(gr.getValue('sys_id'));

        conversation.variables.recommended_group = rec.assignment_group_name;
        conversation.variables.confidence = rec.confidence;
        conversation.variables.reasoning = rec.reasoning;
        conversation.variables.group_sysid = rec.assignment_group_sysid;
    } else {
        conversation.variables.error = "Incident not found";
    }
})();
```

---

# 第六部分：测试与验证

## 6.1 单元测试

### 6.1.1 测试 Script Include

**步骤 1**: 导航 **System Definition → Scripts - Background**

**步骤 2**: 粘贴测试代码：

```
// 测试 AIRoutingEngine
var testIncidentId = '[一个测试incident的sys_id]';
var aiRouter = new AIRoutingEngine();
var result = aiRouter.getAssignmentRecommendation(testIncidentId);

gs.info('Test Result: ' + JSON.stringify(result, null, 2));
```

**步骤 3**: 点击 **Run script**

### 6.1.2 测试 Business Rule

**步骤 1**: 创建测试 Incident

**步骤 2**: 设置 Configuration Item（关联到 CI）

**步骤 3**: 保存并观察： - Assignment Group 是否自动更新 - Work Notes 是否添加了 AI 推荐记录

## 6.2 集成测试

### 6.2.1 测试 Azure OpenAI 连接

```
var r = new sn_ws.RESTMessageV2('Azure OpenAI Service', 'chat_completion');
r.setStringParameter('system_prompt', 'You are a helpful assistant');
r.setStringParameter('user_prompt', 'Hello, are you working?');

var response = r.execute();
```

```
var body = response.getBody();
gs.info('Response: ' + body);
```

### 6.2.2 测试 P1 通知

**步骤 1**: 创建 P1 Incident

**步骤 2**: 验证： - Email 是否发送给 On-Call 工程师 - SMS 是否触发（如果配置了） - Teams 消息是否发送

### 6.2.3 测试 Virtual Agent

**步骤 1**: 打开 Service Portal → Virtual Agent

**步骤 2**: 输入："Who should handle INC0012345?"

**步骤 3**: 验证： - Virtual Agent 是否识别 Intent - 是否返回正确的 Assignment Group 推荐 - 是否可以一键更新 Assignment

---

## 6.3 性能优化

### 6.3.1 添加缓存机制

如果 AI 调用频繁，可以添加 GlideCache：

```
// 在 AIRoutingEngine 中添加缓存
_getCachedRecommendation: function(incidentHash) {
    var cache = new GlideCache();
    cache.setTimeOut(300); // 5分钟缓存
    return cache.get(incidentHash);
},

_setCachedRecommendation: function(incidentHash, recommendation) {
    var cache = new GlideCache();
    cache.set(incidentHash, recommendation);
}
```

### 6.3.2 异步处理

如果 AI 调用慢，使用 Async Business Rule：

```
When: async
Insert: true
Advanced: true
```

---

# 附录：故障排除

## A.1 常见问题

### Q: Azure OpenAI 返回 401 Unauthorized

**A**: 检查 REST Message 中的 api-key header 是否正确

### Q: Business Rule 不触发

**A**: 1. 检查 Business Rule 是否 Active 2. 检查 Condition 是否正确 3. 检查是否有其他 BR 冲突

### Q: AI 推荐不准确

**A**: 1. 检查 KB Articles 质量和数量 2. 优化 Prompt 中的 instructions 3. 考虑 fine-tuning (需要额外 Azure OpenAI 配置)

### Q: Teams 集成失败

**A**: 1. 检查 OAuth 配置 2. 验证 Azure AD 权限是否 granted 3. 检查 Teams Integration 插件是否最新

---

## A.2 监控与日志

### 启用 DEBUG 日志

```
// 在 Script Include 中添加
this.debug = true;

// 在关键位置
if (this.debug) {
    gs.debug('Debug: ' + JSON.stringify(data));
}
```

### 查看系统日志

```
All > System Logs > System Log
Filter: Source contains "AIRoutingEngine"
```

---

**文档完成时间**: 2026-03-08 **版本**: v1.0 **作者**: 富贵虾 (AI Architect)

# Feasibility Analysis

# ServiceNow + Microsoft Copilot 集成方案可行性分析报告

> 生成时间: 2026年3月8日
> 分析范围: ServiceNow Now Assist 与 Microsoft 365 Copilot 集成

---

## 执行摘要 (Executive Summary)

**初步结论: □ 可行，但需满足特定前提条件**

ServiceNow 与 Microsoft Copilot 的集成在技术上是成熟的解决方案，已在 Xanadu/Zurich 版本中正式 GA。然而，实施可行性高度依赖于现有许可证基础、ServiceNow 版本、以及组织的 AI 就绪度。

| 维度 | 评级 | 说明 |
|---|---|---|
| 技术可行性 | □ 高 | 原生支持，配置相对简单 |
| 成本可行性 | □ 中 | 需 Pro Plus 许可，成本增幅显著 |
| 实施难度 | □ 中 | 2-4 周配置期，需多部门协调 |
| 风险等级 | □ 中低 | 数据隐私控制完善，但需主动配置 |

---

## 1. 技术可行性分析

### 1.1 ServiceNow 版本要求

| 版本 | 支持状态 | 功能差异 |
|---|---|---|
| **Xanadu (Patch 1+)** | □ 最低支持 | 基础 Copilot 集成，需自定义 Bot |
| **Yokohama (Patch 8+)** | □ 支持 | 改进的隐私控制 |

| 版本 | 支持状态 | 功能差异 |
|---|---|---|
| **Zurich (Patch 2+)** | □ **推荐** | Custom Engine Agent (CEA)，原生插件支持，响应更快 |

**当前环境评估**: Yokohama □ - 满足最低要求 (Xanadu+) - 支持 Now Assist 基础功能 - 可运行，但 Zurich 版本体验更佳

### 1.2 Azure OpenAI 可用性

ServiceNow Now Assist 的 AI 能力由以下 LLM 提供支持：

| LLM 来源 | 使用场景 | 注意事项 |
|---|---|---|
| **Now LLM** (ServiceNow 自研) | 默认推理 | 数据不用于训练，零持久化 |
| **Azure OpenAI** | 可选备用 | 需额外配置 |

**重要说明**: - Now Assist **不强制要求** Azure OpenAI Service 订阅 - ServiceNow 提供托管的 LLM 服务，数据在客户区域处理

### 1.3 所需插件清单

| 插件名称 | 最低版本 | 用途 | 许可要求 |
|---|---|---|---|
| Virtual Agent | 最新 | 虚拟代理基础功能 | 包含在基础平台 |
| Conversational Integration with Microsoft Teams | 10.1.1+ | Teams 集成 | 包含 |
| Now Assist | 最新 | AI 功能核心 | **Pro Plus 或 Enterprise Plus** |
| Integration Hub | Standard Edition | 事务处理 | $100/月起 |
| Data Privacy for Now Assist | 最新 | PII 脱敏 | 免费插件 |

---

## 2. 许可证成本分析

### 2.1 Now Assist 许可费用

Now Assist 采用 **叠加许可模型**，需在基础许可之上购买 Plus 附加包：

| 基础许可 | Plus 附加包 | 估计成本增幅 |
|---|---|---|
| ITSM Pro | ITSM Pro Plus | **+30%~45%** |
| ITSM Enterprise | ITSM Enterprise Plus | **+30%~45%** |

**许可类型限制**: - □ **支持:** Fulfiller、Business Stakeholder、Unrestricted 用户 - □ **不支持:** Creator SKU (开发者许可)

### 2.2 Microsoft 365 Copilot 许可

| 许可类型 | 成本 (USD/用户/月) | 说明 |
|---|---|---|
| Microsoft 365 Copilot | ~$30 | 标准企业版 |
| Teams + Microsoft 365 | 包含在上述许可 | 必需基础 |

**总拥有成本 (TCO) 估算 (100 用户示例)**:

| 成本项 | 月费用 | 年费用 |
|---|---|---|
| ServiceNow ITSM Pro (基础) | $10,000 | $120,000 |
| Now Assist Plus 升级 (+40%) | $4,000 | $48,000 |
| Microsoft 365 Copilot (100用户) | $3,000 | $36,000 |
| Integration Hub Standard | $100 | $1,200 |
| **总计** | **~$17,100** | **~$205,200** |

---

## 3. 实施难度评估

### 3.1 开发工作量估算

| 阶段 | 工作量 | 时间估算 | 所需角色 |
|---|---|---|---|
| 规划与评估 | 低 | 1周 | 项目经理、架构师 |
| 环境准备 | 低 | 2-3天 | ServiceNow 管理员 |
| 插件安装配置 | 低 | 1-2天 | ServiceNow 管理员 |
| Bot 配置 | 中 | 3-5天 | ServiceNow + Azure 管理员 |

| 阶段 | 工作量 | 时间估算 | 所需角色 |
|---|---|---|---|
| Copilot 集成启用 | 低 | 1天 | ServiceNow 管理员 |
| LLM Topic 配置 | 中-高 | 1-2周 | 流程分析师、开发者 |
| 测试与调优 | 中 | 1-2周 | QA、业务用户 |
| 培训与上线 | 低 | 1周 | 培训师、Change Manager |
| **总计** | | **4-8 周** | |

### 3.2 配置复杂度分析

**简单配置 (开箱即用)**: 使用默认 Bot，启用标准集成 **复杂配置 (需定制)**: 自定义 Bot、LLM Skills、多语言支持

---

## 4. 风险评估

### 4.1 数据隐私 / PII 风险

| 风险项 | 等级 | 缓解措施 |
|---|---|---|
| PII 泄露到 LLM | □中 | 启用 **Data Privacy for Now Assist** 插件，实时脱敏 |
| 数据跨境传输 | □低 | 区域数据驻留，符合 GDPR/CCPA |
| 提示词持久化 | □低 | ServiceNow 零持久化策略，处理完成后立即删除 |

**隐私保护技术**: 1. **零持久化**: 提示词和响应仅内存处理，不存储 2. **端到端加密**: TLS 1.2 加密传输 3. **客户数据隔离**: 逻辑隔离环境

### 4.2 AI 推荐准确性风险

| 风险项 | 等级 | 说明 |
|---|---|---|
| 错误响应/幻觉 | □中 | LLM 可能生成不准确信息 |
| 上下文理解偏差 | □中 | 复杂查询可能理解不准确 |

**缓解策略**: - 定期审查和更新知识库 - 配置反馈机制 - 使用 RAG 确保基于事实的知识 - 高敏感操作保留人工审核

### 4.3 集成故障影响

| 故障场景 | 影响等级 | 缓解措施 |
|---|---|---|
| Copilot 服务中断 | □中 | 用户可直接使用 Teams Bot 或 ServiceNow 门户 |
| ServiceNow 实例故障 | □高 | 依赖 ServiceNow SLA，建议高可用配置 |

---

## 5. 最终结论与建议

### 5.1 可行性结论

| 评估项 | 结论 |
|---|---|
| **技术可行性** | □ **可行** - 原生支持成熟，配置相对简单 |
| **经济可行性** | ⚠ **需谨慎评估** - 许可成本增幅 30-60%，需 ROI 论证 |
| **实施可行性** | □ **可行** - 4-8 周可完成基础部署 |
| **风险可控性** | □ **可控** - 隐私保护完善，但需主动配置 |

### 5.2 针对当前环境的建议

**□ 建议推进，前提是：**

1. □ 当前版本 Yokohama 满足最低要求 (≥ Xanadu)
2. ⚠ 确认已有 **Now Assist Pro Plus** 许可 (或计划购买)
3. □ Copilot Subscription 已确认
4. □ 技术人员到位

**优化建议**: - 如预算允许，建议升级到 Zurich 以获得最佳体验 - 先试点部门验证效果，再全面推广 - 整理知识库内容，提高 AI 回答质量

### 5.3 下一步行动

| 优先级 | 行动项 | 时间 |
|---|---|---|
| P0 | 确认 Now Assist Pro Plus 许可状态 | 本周 |
| P0 | 获取详细报价和年度预算 | 本周 |

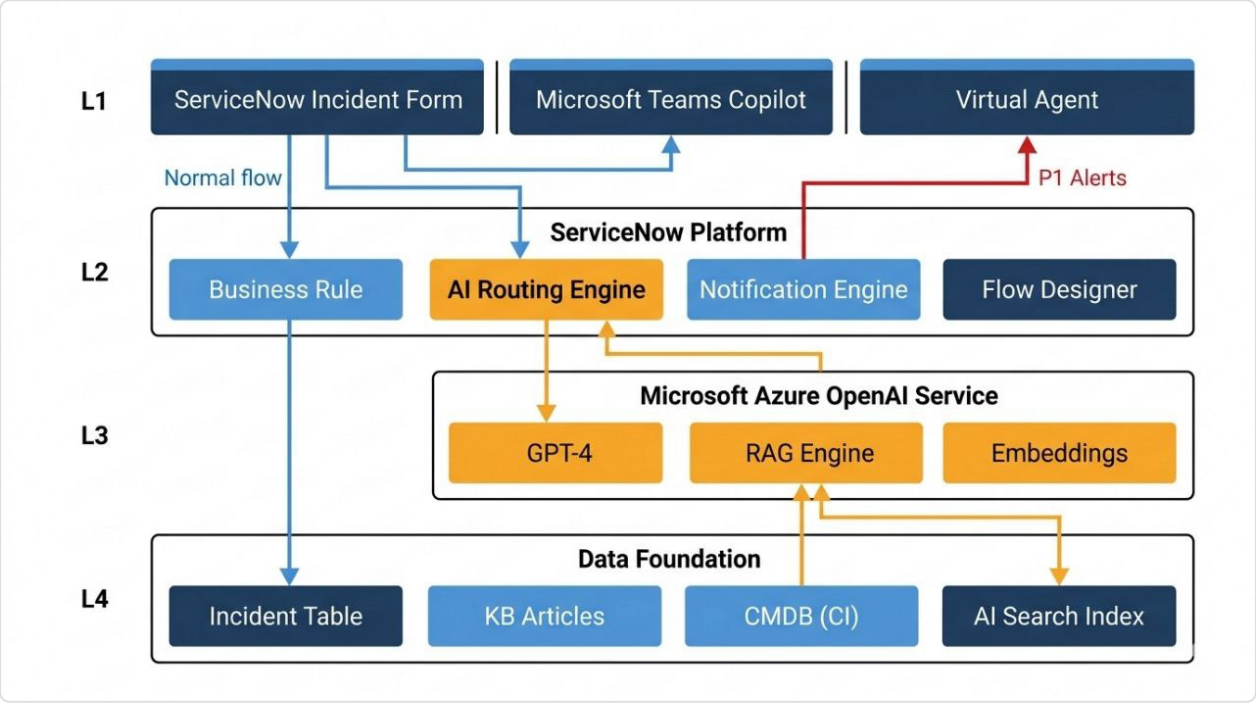
| 优先级 | 行动项 | 时间 |
| --- | --- | --- |
| P1 | 制定试点计划 (选1-2个部门) | 2周内 |
| P2 | 安排管理员培训 | 1个月内 |

---

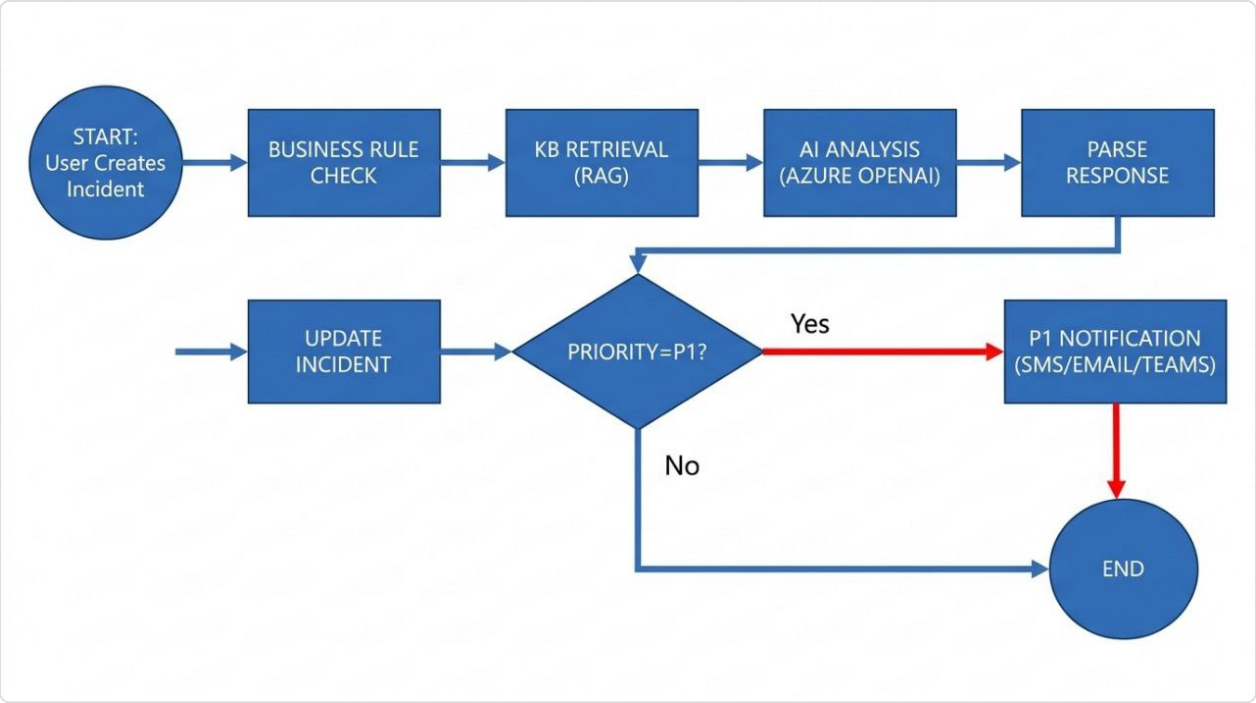
**分析报告完成** | 富贵虾 (AI Architect) | 2026-03-08

# Architecture Diagrams

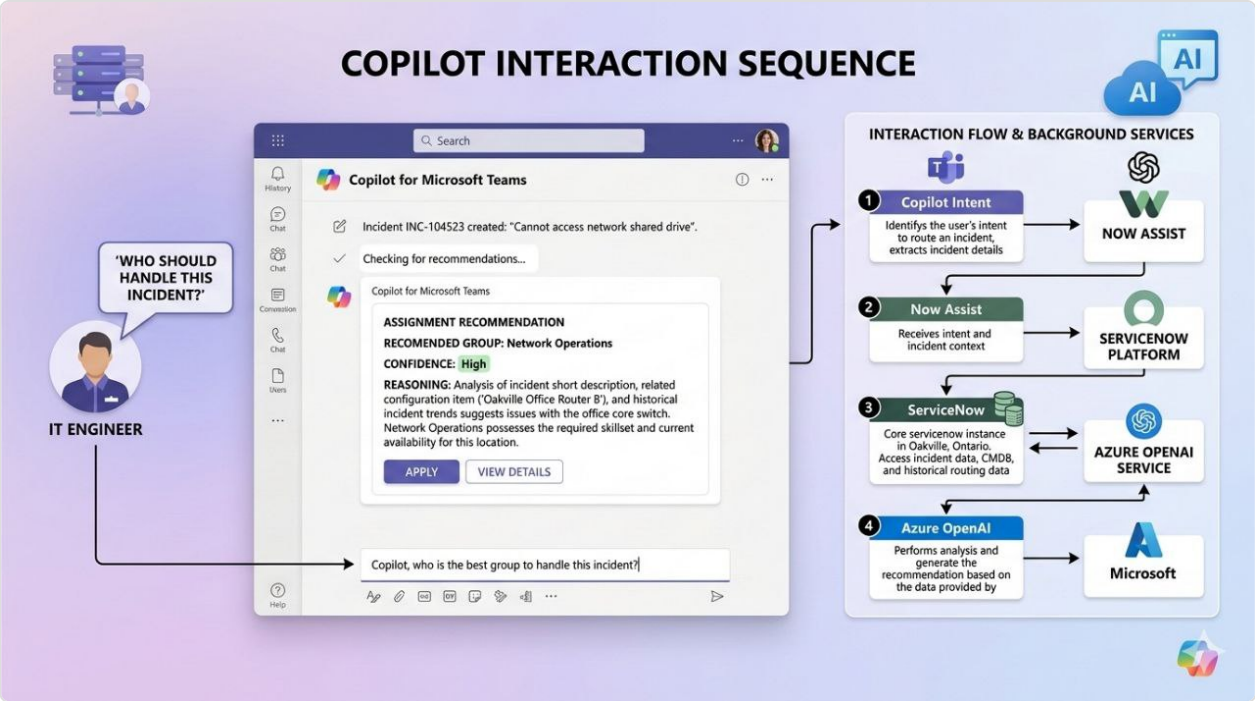
## Overall System Architecture



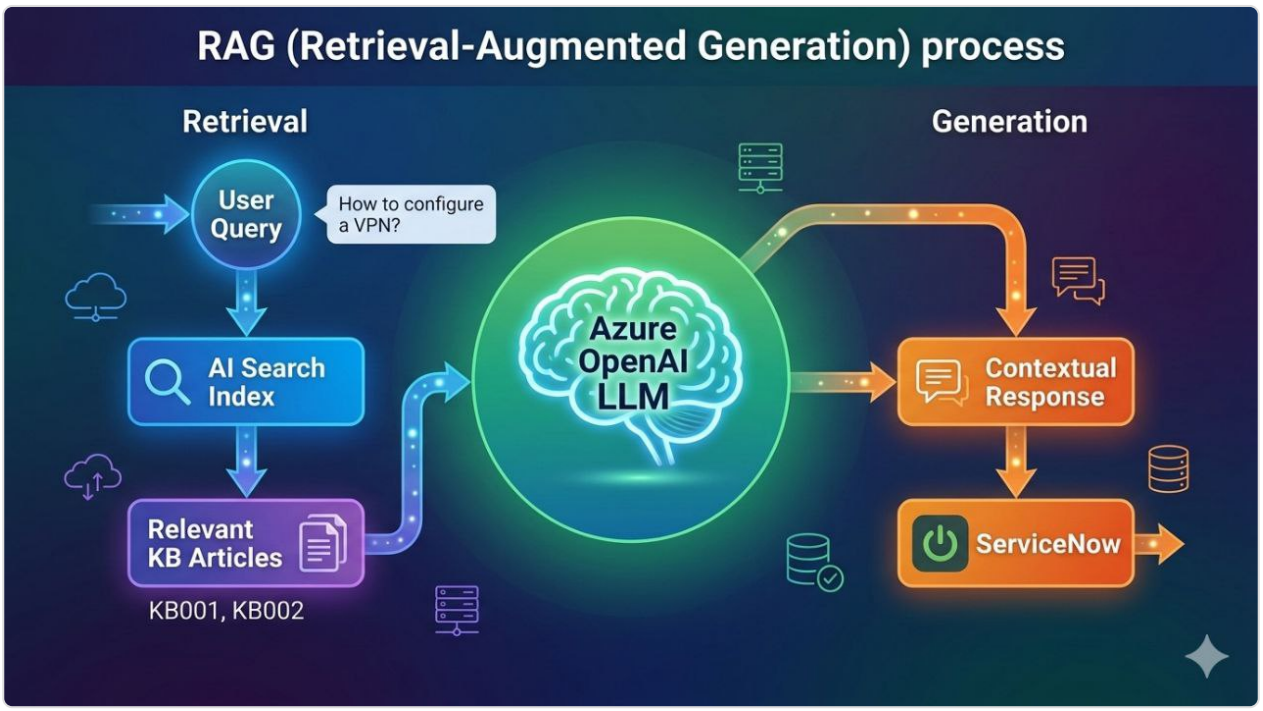
# Incident Creation Data Flow



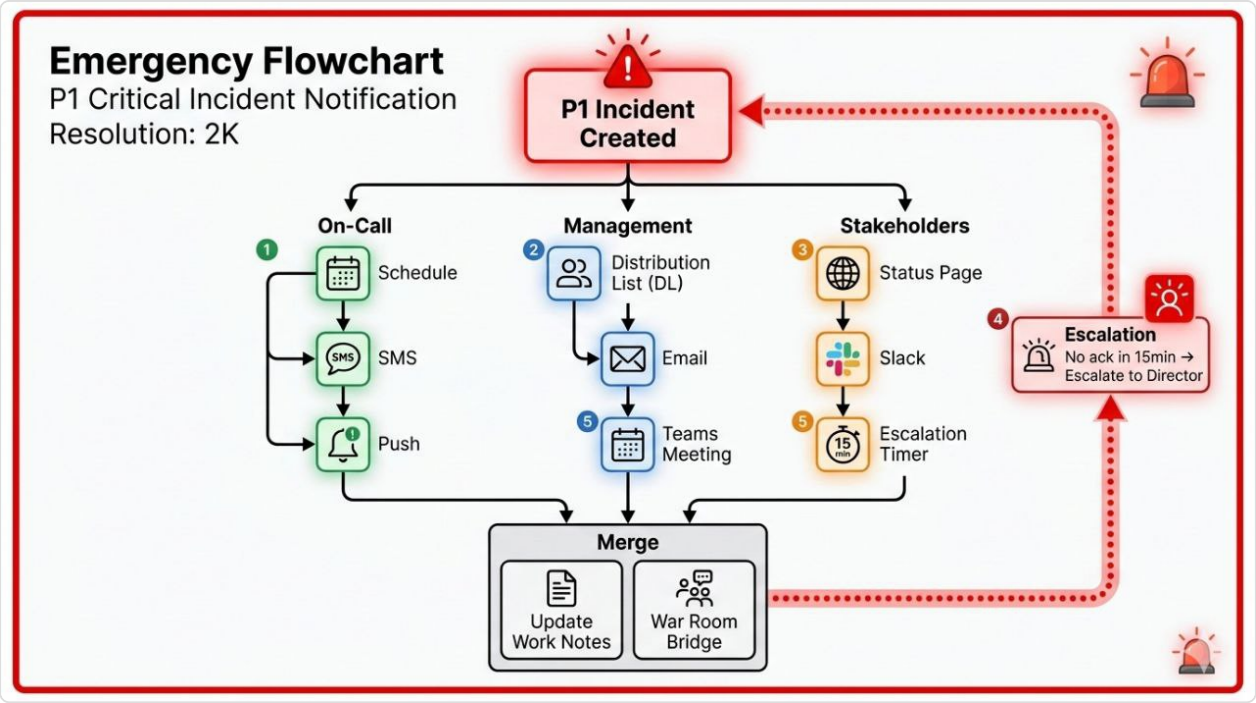
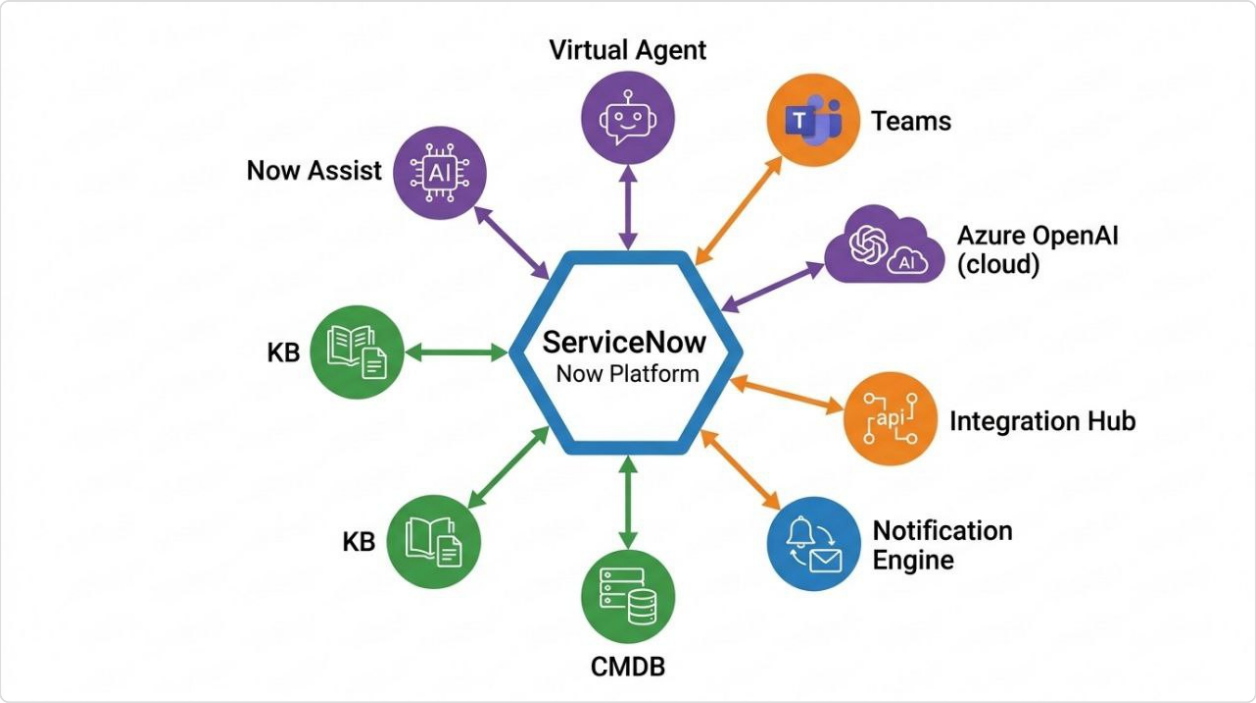
# Copilot Interaction Flow



# RAG Retrieval Process



# P1 Notification Workflow



# Component Relationship